```
          ...                                          ❶
          Behaviors
            .receiveMessage[Command] {
              case LogFile(file) =>
              ...                                      ❷
            }
            .receiveSignal {
              case (_, Terminated(ref)) =>
              ...                                      ❸
            }
        }
      }
      .onFailure[ParseException](SupervisorStrategy.resume)
```

❶ Spawns a database writer with a URL from the settings and watches it

❷ Reads a file, parses it by line, and sends the cleaned line to the database writer

❸ Re-creates the database writer or stops itself

The log processor is created from each file that the file watcher finds in each directory. This log processor is now responsible for creating a database writer, monitoring it, and re-creating it as needed.

Finally, listing 5.6 shows the `DbWriter` actor, the last chain of the hierarchy. This actor receives a parsed row and converts it into the schema required by the database. In case of a restart or stop, it closes its connection to avoid resource leaking. Depending on which exception `UnexpectedColumnsException` or `DbBrokenConnectionException` throws, the actor resumes or restarts. Both strategies are combined in a single function for better readability.

Listing 5.6 `DbWriter` sketch implementation

```
object DbWriter {

  sealed trait Command
  final case class Line(time: Long, message: String, messageType: String)
      extends Command
```